

C语言与程序设计

The C Programming Language

第2章 基本词法语法规则与程序元素

王多强

1097412466

dqwang@hust.edu.cn

华中科技大学计算机学院

学习一门语言，要关注两方面的问题：语法和语义

语法：指语言的文法结构，要求按语言指定的规则组词、造句

—— 写作上的正确性，不能写错别字、病句。



语义：指词、句的含义，在文法正确的基础上是否正确表达了逻辑含义 —— 逻辑上的正确性，表达出正确的意思。



2.1 如何描述计算机语言的语法规则？

人类的每种语言都有其基本的语法规则：

- ◆ 字 (字母)
- ◆ 词
- ◆ 句：主-谓-宾

计算机语言也有它的语法规则：字、词、句的结构

- 相对人类的自然语言，计算机语言有更严格和死板的语法规则定义，其描述工具有以下几种：
 - ◆ BNF范式
 - ◆ 语法图

2.2.1 BNF 范式（巴科斯范式）

BNF (Backus-Naur Form) 是用来描述计算机语言语法的符号集和规则，如：

<小写字母> ::= a|b|c|d|...|z

<大写字母> ::= A|B|C|D|...|Z

<数字> ::= 0|1|2|3|4|5|6|7|8|9

<字母数字> ::= <字母>|<数字>

<字母> ::= _|<大写字母>|<小写字母>

<标识符> ::= <字母>|<标识符><字母数字>

BNF范式由 John Backus 和 Peter Naur 首先引入的。现在，几乎每一位编程语言书籍的作者都使用BNF范式来定义编程语言的语法规则。

自学BNF范式和语法图等

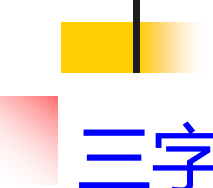
2.2 C语言的词法元素

C程序（这里特指**源程序**）本质上是一个字符序列，其中的任何**词法元素**都是由**字符**组成。

2.2.1 字符集

C语言的字符集为**ASCII字符集中的所有可打印字符及少量控制字符**：

- **英文字母**：a~z 和A~Z
- **数字字符**：0~9
- **特殊字符**：! " # % & ' () * + , - . / :
(英文标点符号) ; < > = ? [] \ ^ _ { } | ~
- **空白字符**：空格、换行符、制表符等



三字符序列（自学，教材表2-1, p21）：

以两个连续问号开头的三字符序列(教材表2-1, p21)。

所有的三字符序列都要用相应的单个字符替换，这种替换发生在其他任何处理之前。

例如，

`int a??(4??)=??<0??>;` 被替换成

`int a[4]={0};`

2.1.2 词法元素

C源程序是一个**字符序列**。

```
int main(void)
{
    char name[20];
    printf("Input your name please!\n");
    gets(name);
    printf("Hello %s!\n",name);
    show(name);
    return 0;
}
```

- ◆ 编译器在编译时，首先对C源程序中的字符序列进行分割——**分词**，形成一个个具有语义成分的基本单元（单词和运算符）。
- ◆ 这些基本单元称为**记号**(Token)，是程序的基本组成元素，称为**词法元素**。

例2.1 `sum=x+y`

分解成`sum`、`=`、`x`、`+`和`y` 共5个记号。

其中`sum`、`x`、`y`是标识符，`=`、`+`是运算符

例2.2 `int a, b=10;`

分解成`int`、`a`、`,`、`b`、`=`、`10`和`;` 共7个记号

其中`int`是关键字，`a`、`b`是标识符，`=`是运算符，`10`是常量

例2.3 `x+++++y`

分解成`x`、`++`、`++`、`+`、`y` 共5个记号

◆ C语言中记号共分5类:

- 标识符
 - 关键字
 - 常量
 - 运算符
 - 标点符号
- 词
- 符号

◆ C语言中词、句的构造规则是什么样的呢?

—— C语言的语法规则

2.3 词的构造：标识符、关键字

2.3.1 标识符

- ◆ C语言中所有的名字都用“标识符”标注

- 如：常量名、变量名、数据类型名和函数名等。
- 标识符是C语言中的单词，是一个字符序列

- ◆ 标识符的命名规则

- 以字母或下划线开头，后跟字母、下划线或数字的组合。

这里，字母是指：a~z, A~Z, 并且是大小写敏感的

数字是指：0~9

下划线是：_

例：以下哪些是合法的标识符：

K



k123



_id



_id



month



time1



20_sum



not#me



标识符的命名规则：

以字母或下划线开头，后跟字母、下划线或数字的组合

C语言是大小写敏感的，即区分字母的大小写。

如：Name、name、name 表示不同的单词

_20_sum



_not#me



友情提示：

良好的编程风格是选择**有助于记忆且有一定含义**的标识符，
这样可增强程序的可读性和可维护性。

被大家认可的命名法有：**匈牙利命名法、驼峰式命名法、帕斯卡命名法等**。（自己查找资料，课外作业）

例：**匈牙利命名法**是一种编程时的命名规范。基本原则是：

变量名 = 属性 + 类型 + 对象描述

其中对象的名称要有明确含义，可以取对象名字全称或名字的一部分。要基于容易记忆、容易理解的原则。保证名字的连贯性是非常重要的。如：int iMyAge; float fManHeight

- **驼峰式命名法**：第一个单词首字母小写，后面其他单词首字母大写。 如：int myAge;
- **帕斯卡命名法**：每个单词的第一个字母都大写。 如：int MyAge;

2.3.2 关键字

关键字：是一种被系统预先赋予了**特定含义**和**专门用途**的**标识符**，
是系统的“**专有名词**”，又称为**保留字**。

C90标准里定义了32个关键字

auto	break	case	char	const	continue	default	do
double	else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

C99标准新增5个关键字



inline	restrict	_Bool	_Complex	_Imaginary
--------	----------	-------	----------	------------

C11标准新增7个关键字

_Alignas	_Alignof	_Atomic	_Static_assert	_Noreturn	_Thread_local	_Generic
----------	----------	---------	----------------	-----------	---------------	----------

详见教材表2.2 (p24)

- C程序里除关键字外的其它标识符称为 “**用户自定义的标识符**”
- 特别注意：用户自定义的标识符不要和关键字重名。

2.3.3 分隔符

- ◆ 不同的标识符之间要分开

如, `int x,y;` 不要写成 `intx,y;` 或 `intxy;` 否则含义完全不同。

- ◆ 最常用的分隔符：空格

- 其它还有：制表符、换行符、注释符等。

- ◆ 分隔符在语法上起分隔单词的作用

- 当两个相邻的单词之间如果不用分隔符就不能区分开时就必须加分隔符
(分隔单词除了用分隔符, 还可以用运算符及其他符号等)。

- 通常, 多加分隔符对程序没影响, 编译器会忽略掉多余的分隔符。

2.4 数据和数据类型

- 现实世界中的各种信息在计算机中是以**数据**的形式存在的，数据是程序处理的对象。
- 数据有类型之分

现实世界中的数据有：

- ◆ **数值型**数据：整数、实数（注：计算机不能直接处理复数）
- ◆ **字符型**数据：文本、字符序列
- ◆ **其他数据**：图像、音频等（这些数据最终都转换成数值型数据输入到计算机系统中进行处理）

其中，数值型和字符型是计算机能够处理的基本数据类型。

- 数据类型是一种语言的数据表达能力
 - 类型越丰富，对客观世界的描述能力越强大
 - C语言提供了丰富的数据类型

2.4.1 C语言的基本数据类型

C语言的基本数据类型只有三种：

- ◆ 表示整数的类型：称为**整型**
- ◆ 表示实数的类型：称为**浮点类型**
- ◆ 表示字符的类型：称为**字符型**

1. 数据类型的基本知识

(1) 数据类型规定了什么呢?

规定了这种类型的数据的

- 取值范围
- 可参与的运算
- 内存中存储方式

其中，取值范围和存储方式都与数据的内存表示相关

(2) 数据类型的长度

计算机采用的是二进制，每个bit可取0或1，用多个bit，根据组合关系，就可表示更大的数：

- 1bit可表示： $(0\sim 1)_{10}$ ，即 $(0, 1)_2$
- 2bit_s可表示： $(0\sim 3)_{10}$ ，即 $(00, 01, 10, 11)_2$
- 3bit_s可表示： $(0\sim 7)_{10}$ ，即 $(000, 001, 010, 011, 100, 101, 110, 111)_2$
- 4bit_s可表示： $(0\sim 15)_{10}$ ，即 $(0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000, 1001, 1010, 1011, 1100, 1101, 1110, 1111)_2$

以此类推,

➤ 8bit_s可表示: $(0 \sim 255)_{10}$, 即 $(00000000 \sim 11111111)_2$

➤

- ◆ 若用n-bit_s表示整数, 可表示的数据范围是 $0 \sim 2^n - 1$, 对应的二进制表示就是:

$$\underbrace{(000 \dots 0)}_{n \text{ 个}}_2 \sim \underbrace{(111 \dots 1)}_{n \text{ 个}}_2$$

- ◆ 若一个数据用n-bit_s表示, 亦称这个数据的长度为n位
- ◆ 但由于计算机中位不能单独存取, 最小存取单位是字节, 所以一般用数据占用的字节数表示数据的长度。

- ◆ 若有N个字节，则总共有8N位，可表示的数范围是 $0 \sim 2^{8N} - 1$
 - 显然，字节数越多，表示的数范围越大；
 - 但内存是有限的，以4GB为例，内存总共有4G个字节，数据表示范围： $0 \sim 2^{32G} - 1$ ，也是有限的；
 - 事实上，不可能用整个内存空间表示一个大整数的；
 - 同时，再大的内存也表示不了现实世界所有的数。

◆ 怎么办？

通常，计算机只是用有限的几个字节表示某种数据类型的数据。

比如，字符型数据用1个字节表示

整型数据用2个或4个字节表示

浮点数据用4个或8个字节表示

(4) 数据类型的名词

- ◆ 每种类型都有一个**类型名**

- 类型名是**关键字**，通常取自该类型的英文名或“简称”

- ◆ **类型修饰符**

- 在说明数据类型的时候，可以将**类型修饰符**放于类型名的前面，用于从基本类型**派生**出的扩展类型。

- 常用的类型修饰符有：

- ◆ 表示**有符号数据**：signed
- ◆ 表示**无符号数据**：unsigned
- ◆ 表示“短”整型数：short
- ◆ 表示“长”整型数：long

用法及含义见后

2. C语言基本类型1——字符型

- 基本类型名：**char**
- 数据的表示形式：用一对英文单引号括起来单个字符（或转义字符序列），如：**'a'**、**'1'**、**'@'**
- 存储长度：**1个字节**
- 数据的值：**字符的ASCII码**，本质上是个整数
 - 可以当作“小”整数看待，取值范围：0~127
如： $5 + 'a'$ 等价于 $5 + 97$
 - 可以把整个字节都用来表示“小整数”（即视为8bit_s的整数）
 - 无符号字符：**unsigned char**，取值范围0~255。
 - 有符号字符：**signed char**（或**char**），取值范围-128~127。

3. C语言基本类型2——整型类型

整型数据用于表示有符号或无符号整数，有以下基本形和

扩展形：

- ◆ **int**：基本形；相当于 **signed int**；长度等于**机器字长**，16位机int型为2字节，32位机int型为4字节。
- ◆ **short int**：“短”整型，长度一般为2个字节。
- ◆ **long int**：“长”整型，理论上长度为int的2倍，如16位机长整型长度为4字节，32位机long int型也为4字节
- ◆ **unsigned int**：无符号整型
- ◆ **unsigned short int**：无符号短整型
- ◆ **unsigned long int**：无符号长整型

整型数据的取值范围

(1) 设机器字长为2B (2Byte, 16位) , 则相应取值范围为:

- **int** : -32768~32767,
- **unsigned int** : 0 ~ 65535
- **long int** : -2147483648 ~2147483647,
- **unsigned longint** : 0 ~ 4294967295

(2) 设字长机器为4B (32位) , 则相应取值范围为:

- **int** : -2147483648 ~2147483647,
- **unsigned int** : 0 ~ 4294967295

4. C语言基本类型3——浮点类型

浮点类型有以下3种：

- ◆ **float**：单精度，4字节长度
- ◆ **double**：双精度，8字节长度
- ◆ **long double**：（c99新增类型，自学）

浮点数表示要解决三个问题：

- ◆ 正负号
- ◆ 尾数
- ◆ 阶码

这是一个世界级的问题！

浮点数的表示:

- 一个浮点数V可表示为以下形式 (类似科学计数法)

$$V = (-1)^{\underset{\substack{\downarrow \\ \text{数符}}}{S}} \times M^{\underset{\substack{\downarrow \\ \text{尾数}}}{}} \times 2^{\underset{\substack{\downarrow \\ \text{指数}}}{E}}$$



Prof. William Kahan

发明者

1989
ACM Turing
Award Winner!

编码规则: IEEE754标准, 规定了单精度、双精度浮点数的编码规则

数符: 用1位表示, 表示数的正、负;

尾数: 采用**规格化形式**: 1.XXXXXX, 但只存储XXXXXX部分, 1省略,
用23位 (单精度) 或52位 (双精度) **原码表示**;

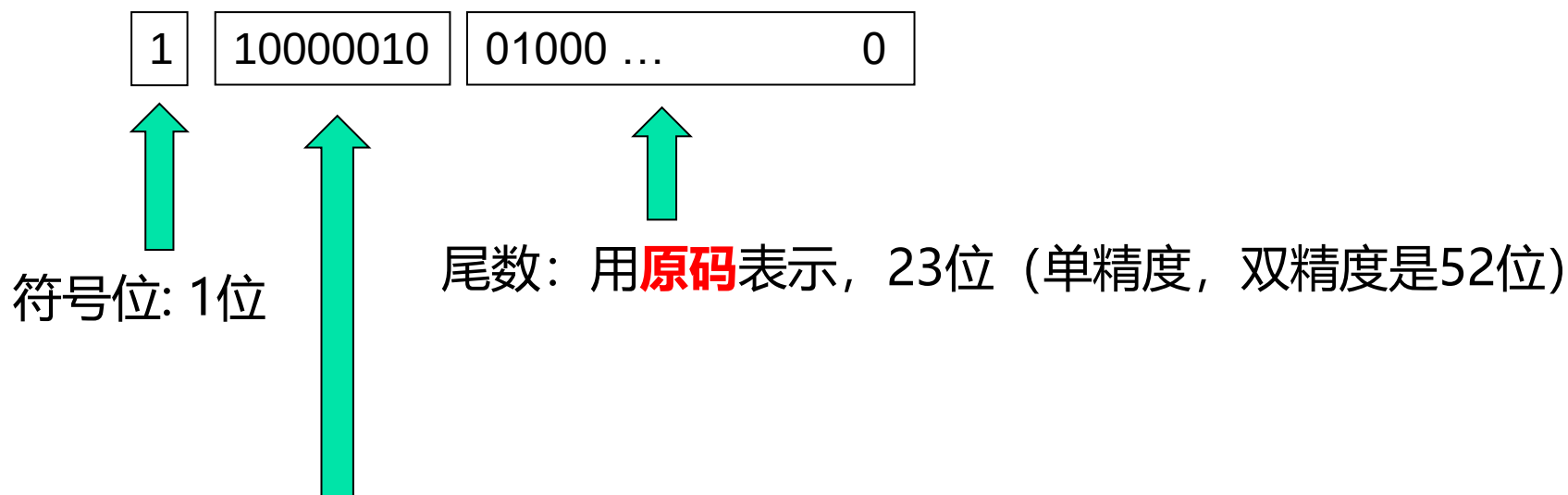
指数: 代表2的幂指数, 用8位 (单精度) 或11位 (双精度) **移码表示**。

■ 如: $(-10.0)_{10} = (-1010.0)_2 = (-1.01 * 2^3)_2 = ((-1)^1 * 1.010 * 2^3)_2$

非规格化尾数

规格化尾数

标准形式



阶码: 用移码表示, 8位 (单精度) 或11位 (双精度)

阶码 = 偏置常数 + 指数

偏置常数: 单精度127 或 双精度1023

《计算机系统基础》课程里再做详解

- 尾数的位数决定值的精度，指数的位数决定值的范围

- float: 4B, 其中符号1b, 指数8b, 尾数23b

- 其精度大约为7位十进制数

- 范围约 $10^{-38} \sim 10^{+38}$ 。

- double: 8B, 其中符号1b, 指数11b, 尾数52b

- 其精度大约为15位十进制数

- 范围约为 $10^{-308} \sim 10^{+308}$ 。

- 很多编译器将long double处理为double，在某些系统中，它占用10或12B。很少被使用。

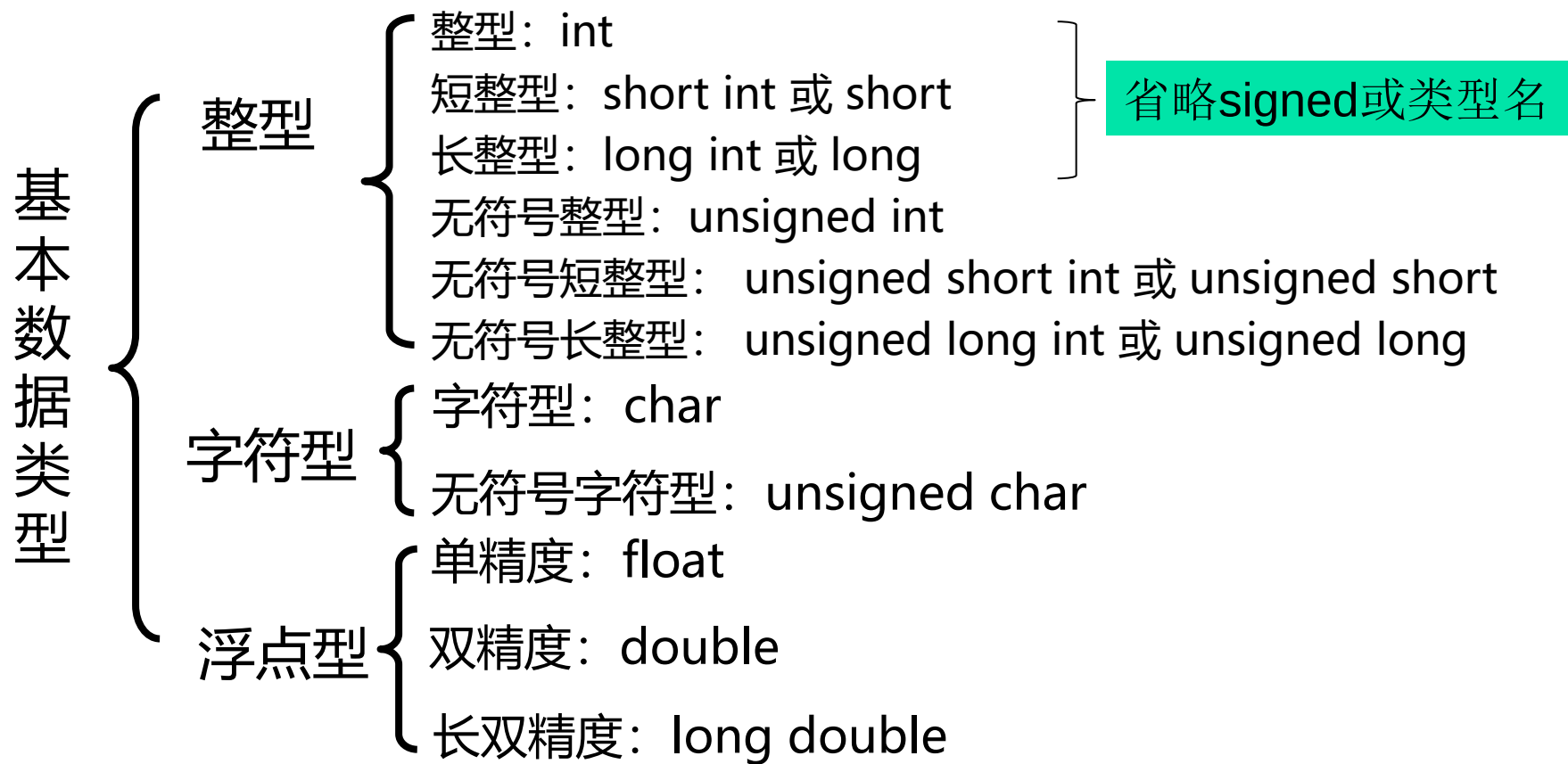
- 尽管有特定的格式，但浮点数的表示本质上还是0/1组合，因此计算机内表示的浮点数是有限个数的离散数据，不是连续和无限的。因此大多数情况下，浮点数的表示可能只是近似的（查阅资料自学）。

如：1.001001001...

- 其实际值与表示法之间的差称为“可表示误差”。
 - 计算也可能造成可表示误差。
- 不能使用==和!=运算符直接比较浮点数据。

解决方案：可以用两个数值之差同一个小正数epsilon比较的方法解决这个问题。

经过基本数据类型名和类型修饰符“组合”后，可能的基本数据类型有：



注：每种类型有不同的字节长度和可表示的数据范围，应根据应用的需要选择合适的类型表示数据。

编程作业： sizeof运算符的使用

运算符sizeof可以求出一种数据类型的大小

用法： sizeof(int)、 sizeof(unsigned long)、 sizeof(double) 等；

任务： 编程查看各种基本数据类型的大小

输出方式：

```
printf("sizeof(int)=%d\n", sizeof(int));
```


依然存在的问题：

基本数据类型的数据只能表示一个值：一个字符、一个整数或一个浮点数，但

- ◆ 一组字符（或一组整数，一组浮点数）怎么表示？

- 定义数组

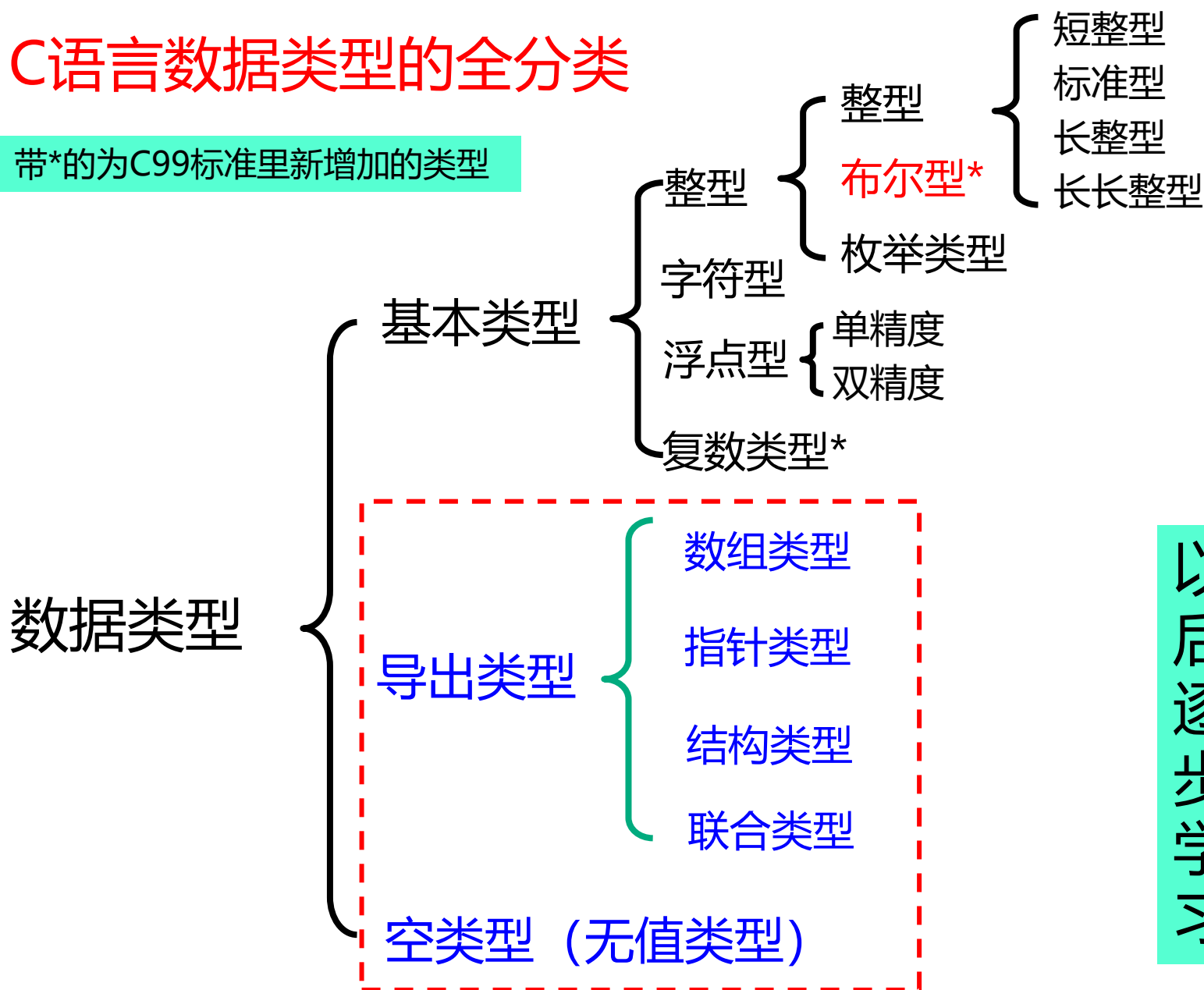
- ◆ 不同类型的数据怎么组成一个“集合”？

- 结构型数据

以后逐步学习

C语言数据类型的全分类

带*的为C99标准里新增加的类型



2.5 常量与变量

- ◆ 程序执行期间值保持不变的数据称为常量;
 - ◆ 程序执行期间值可以改变的数据称为变量。
 - ◆ 这里先研究： 常量
- 常量的定义有三种方式：
- ◆ 文字常量
 - ◆ 符号常量
 - ◆ “变量形式” 定义的常量

2.5.1 文字常量

文字常量是指在源程序中直接写出来的常量；

如：int x=12345;

float f = 3.14;

char c= 'a' ;

char name[20]= "Zhang Hua" ;

- ◆ 文字常量有数值型常量和字符型常量
 - 数值型常量又分为整型常量、浮点型常量
 - 字符型常量又分为字符常量、字符串常量

1. 整型常量

- ◆ 整型常量在程序中代表一个整数
- ◆ 根据进位制的不同，有三种表示方法，通过前缀字符区分：
 - 十进制：无前缀，如：12345
 - 八进制：前缀为0，如：012345
 - 十六进制：前缀为0x或0X，如：0x12345 或 0X12345

例：十进制的31可写成 **037**、**0x1f**、**0X1F**

八进制

十六进制

后缀：整型常量可以带有**后缀**，用以表示扩展类型：

后缀符号	含义	举例	备注
u或U	unsigned型整数	5u 或 5U	
l 或 L	long型整数	5l 或 5L	
ul 或 UL	unsigned long型整数	5ul 或 5UL	
ll 或 LL	long long型	5ll 或 5LL	C99以上标准支持
ull 或 ULL	unsigned long long型	5ull 或 5ULL	C99以上标准支持

注：

- ◆ 无后缀时，仅表示int。
- 当常量值超出指定类型的范围时，其实际类型取决于数值大小、前缀等，
如：int x = 10000000000000000; **x=?**
- 确定类型的规则很复杂，标准化前的C语言、C89和C99中各不相同。

2. 浮点型常量

- ◆ 浮点型常量在程序中表示一个实数

根据写法，有两种表示方式：

(1) **带小数点的十进制数**形式， 如：23.7, 14., .126

注：可以小数点开头，也可以小数点结尾，但整数部和小数部不能同时为空。

(2) **指数形式**：科学计数法

形式：**尾数e指数**，如：45e-3 表示 45×10^{-3} ， .15e5 表示 0.15×10^5

- 尾数部分的书写规则与第一种相同，但可以没有小数点，
- 指数部分形式：**e(E)±n**，代表 $10^{\pm n}$ 。注：n必须为整数。

浮点后缀：浮点型常量也可以使用后缀来指定扩展类型

- 后缀f或F：float， 如：0.1f， 或0.1F
- 后缀l或L：long double。

注：无后缀时默认为double型数据（8字节存储）

练习： 1.f、 1.2e-123L、 10E10L ✓
E+10、 10e1.5、 1.2eF ✗

3. 字符常量

- ◆ **字符常量**在程序中表示**单个字符**（本质是这个字符的**ASCII码值**）
- ◆ 字符常量有两种表示形式：

(1) 用**一对单引号**括起一个字符，如： ‘a’， ‘A’， ‘1’ ✓

- 单引号是字符常量定义的**限界标志**，称为**定界符**。

注：单引号本身不是字符数据的一部分

- 必须用一对英文单引号
- 只能包含一个字符

‘abc’ 、 “a” ✗

错误的写法

字符串，不是字符

(2) 转义字符

◆**转义序列**：以 \ 开头的一种特殊字符序列称为**转义序列**。

基本形式：\XXX

其中：\ 是**前导符**，表示转义序列开始；

XXX 有以下两种形式：

(a) **字符转义序列**：\X，其中X是一个字符

➤ \X 整体被转义成另一字符，通常是**控制字符**或和特殊的**图形**、**非图形符号**。

如：'\n'：表示**换行符**

'\t'：表示**水平制表符**

'\0'：表示**ASCII为0的那个字符**
(值为0)

注：为了表示字符，依然要用**单引号**将转义序列括起来

(b) 数字转义序列:

\ 后面跟一个1~3位的八进制数: \ooo, 或

\x 后面跟一个1~2位的十六进制数: \xhh

➤ 其中, **数的大小=需要表示的字符的ASCII值**

【大小不能超过 $(255)_{10}$, 否则报错 (一般为警告)】

例如, 字符 'A' (ASCII值=65) : '\101'、'\x41'

水平制表符'\t': '\11'、'\011' 或 '\x9'、'\x09'

注: 为了表示字符, 依然要用
单引号将转义序列括起来

关于使用转义字符的注意事项参见教材P30

◆ 字符常量小结：

以字符A为例，可以表示为：

- 用一对单引号括起来的基本形式： 'A'
- 用八进制数表示的转义字符： '\101'
- 用十六进制数表示的转义字符： '\x41'

提示：任何情况下，表示一个字符都不要少了单引号

4. 字符串常量

- ◆ 多个字符的序列称为**字符串**
- ◆ **字符串常量**是用一对**双引号**包括起来的**0个或多个**字符。

如: **"string"** 包含6个字符的字符串

"" 包含**0**个字符的字符串, 称为**空字符串**

- ◆ 字符串中可以包含**转义字符**

如: **"string\n"** 包含7个字符, 最后一个转义字符 **\n**

"c:\tc" 包含4个字符, 第三个字符是转义字符 **\t**

需要讨论的高级话题：

1. 如何表示反斜杠\?

`\` 对吗?

错误，无法正确释义，正确的表示：`\\`

2. 如何表示单引号 ' ?

`'` 对吗?

错误，无法正确释义，正确的表示：`\'`

3. 如何表示双引号 " ?

正确，也可以表示为 `\"`

`""` 对吗?

类似的还有：`"""`，也可以表示为 `\\"`

4. 比较 `"3'40"` 和 `"3'40\""` 的不同

5. `printf( "c:\\tc" );` 在屏幕上输出什么?

如果是 `printf( "c:\tc" );` 呢

◆ 字符串的存储：C语言中用**字符数组**来保存字符串常量

如：设有定义：`char name[20] = "C Language";`

则内存中字节表示为：

变量

常量

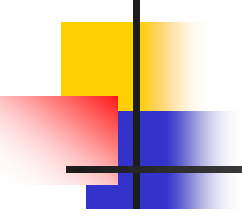
0	1	2	3	4	5	6	7	8	9	10		19
C		L	a	n	g	u	a	g	e	\0		

数组name的字节内容



- C语言的特殊处理：在存储字符串时，系统自动在后面补上一个**空字符**（**\0**，ASCII值为0）作为**字符串结束标志**。
- 因此，**字符串的存储长度比字符串的实际长度要大1**。

思考：如果定义为：`char name[10] = "C Language";` 对吗？



思考： 'a' 与 "a" 有何区别？

'a' ： 字符常量， 占1B内存空间

"a" ： 字符串常量， 占2B内存空间

a	\0
---	----

◆ 如果字符串太长，一行写不下怎么办？

◆ 有两种方法：

(1) 行连接：在前一行的末尾输入**续行符**（\）再换行。

如： "Hello,\

how are you " /* 换行后应紧靠行首 */

(2) 字符串连接：将**字符串分段**，分段后的每个字符串用双引号括起来。

如： "Hello, "

"how are you" /* 换行后不必紧靠行首*/

2.5.2 符号常量

- ◆ 在程序中，可以为一个文字常量起个名字，这个名字称为该文字常量的符号常量；

例：`#define PI 3.141592653589793`

好处：源程序中所有用文字常量的地方都可以用它对应的符号常量代替，简单、直观，减少出错。

如：`PI*r*r`

`3.14159265358979323846 *r*r`

- ◆ 符号常量一定要是一个合法的标识符。

◆ C语言中有三种定义符号常量的方法：

(1) 用`#define`指令

(2) 用`const`声明语句

(3) 用枚举类型（在2.9节介绍）

1. 用#define定义符号常量

- ◆ #define是一种**编译预处理指令**
- ◆ 用#define定义符号常量的格式为：

#define 标识符 常量



符号常量的名字（习惯：一般用大写，以区分变量，但不是必须的）

如：#define PI 3.141592653589793

- 符号常量的使用和处理：**文字代换**，在编译**预处理**阶段，将所有符号常量的出现替换成它所代表的常量（第六章讲述）。

例 输入圆的半径，然后计算圆的面积

```
#include <stdio.h>
```

```
#define PI 3.1415926
```

```
int main(void)
```

```
{
```

```
    int r;
```

```
    float a;
```

```
    printf("Please input radius:");
```

```
    scanf("%d",&r);
```

```
    printf("Radius = %d\nArea=%f\n" , r, PI*r*r);
```

```
    return 0;
```

```
}
```

```
Please input radius:5
Radius = 5
Area=78.539816
```

```
Process returned 0 (0x0)   execution time : 1.796 s
Press any key to continue.
```

定义符号常量的好处：

- 符号常量使常量的含义非常明确
- 使程序易于理解和便于修改

“%d”：格式串，表示要输入一个整数
&r：取变量r的地址，输入的整数就放在这个地址开始的字节单元中

有一定编程基础的同学自学下面的程序，否则等学完循环后再回头看

例2.6 打印华氏和摄氏温度对照表，

温度转换公式为： $^{\circ}\text{C} = (5/9)(^{\circ}\text{F} - 32)$

```
#include <stdio.h>
```

```
#define LOWER 0      /* 温度的下限 */  
#define UPPER 300    /* 温度的上限 */  
#define STEP 20      /* 步长*/
```

```
int main(void)  
{  
    int fahr;
```

```
    for (fahr=LOWER; fahr<=UPPER; fahr=fahr+STEP)
```

```
        printf("%3d:%10.2f\n" , fahr, (5.0/9)*(fahr-32));
```

```
    return 0;
```

```
}
```

\源程序\ex2_6.c

进一步体会定义符号常量的好处：

- 这些名字使常量的含义非常明确：上限、下限、步长。
- 必要的时候也方便修改。
- 符号常量使程序易于理解和便于修改

2. 用const定义符号常量

- ◆ const是关键字，称为类型限定符。

类型限定符：和类型修饰符类似，放在类型之前，表示对所修饰类型的一种扩展性质。

- ◆ 用const声明常量的格式为：

const 类型名 标识符 = 常量;

例如：const double PI=3.14159;

const int DOWN=0x5000;

const int YES=1, NO=0;

- ◆ 用const声明常量时必须初始化。思考为什么？

用const和#define定义的符号常量的区别？

- **const**声明的标识符实际上是一个只读变量

编译时系统会根据定义的类型为该标识符分配存储单元，并把对应的常量值放入其中。但该值不能再被更改（故视为常量）。

此后，程序中每次出现该标识符都是对所代表存储单元的只读访问。

如： **const** double PI=3.14159;

double Area = 2*PI;

- **#define**定义的标识符没有对应的存储单元，只是在编译之前由预处理程序进行简单的文本替换。

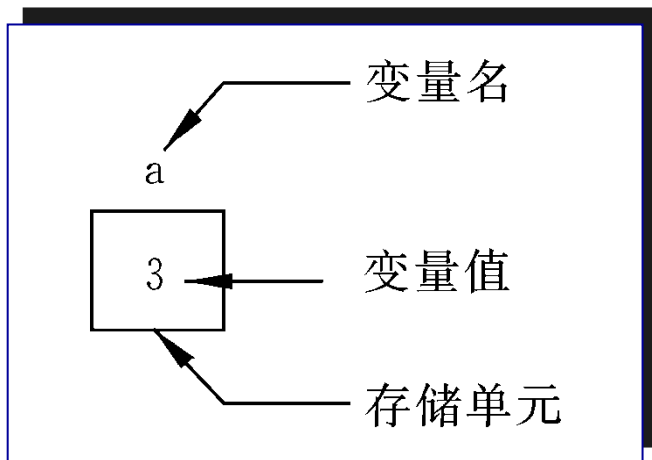
如： **#define** PI 3.14159

double Area = 2*PI;

2.5.3 变量

- ◆ 程序中值可以改变的量称为变量
- ◆ 程序中的变量是一个名称，代表数据在内存中占用的字节单元。

变量可看做是这块内存区域的“有名表示”。



- 变量对应存储单元
- 存储单元中存放了数据
- 数据的大小就是变量的值
- 引用变量一般就是取其值

2.6 运算符和表达式

- ◆ 表示运算功能的符号称为**运算符**

如：**算术运算符**： $+$ 、 $-$ 、 $*$ 、 $/$ 、 $\%$

关系运算符： $<$ 、 $<=$ 、 $==$ 、 $>=$ 、 $>$ 、 $!=$

- ◆ 被运算符操作的对象称为**操作数**
- ◆ 由运算符和操作数组成的有意义的式子称为**表达式**。

如： $\text{sqrt}(b*b-4*a*c)$

$x=x*PI/180$

$\text{fabs}(a_n) \geq \text{EPS}$

x



单个操作数也是表达式的一种形式

2.6.1 运算符基础

1. 对每一种运算符都要明确以下四点：

(1) **功能**：做何种运算

(2) **要求**：操作数的个数和类型

◆ **个数** {
 1个操作数：单目运算符 或 一元运算符
 2个操作数：双目运算符 或 二元运算符
 3个操作数：三目运算符 或 三元运算符

◆ **类型**：一般，运算符对操作数类型有要求

如：%的操作数只能是整数，不能为浮点数。

(3) **优先级和结合性**

(4) **结果类型**：计算结果是什么类型

2. 表达式的类型

- ◆ 每个表达式都有一个运算结果，如：2+3
- ◆ 运算结果是一个具有确定类型的值，这个类型称为**表达式值的类型**，简称**表达式的类型**。
- 当两个**不同类型的操作数**进行运算时，会引起结果数据类型的变化和转换，这时要特别注意结果值的类型。

如：2+3 结果为5，**整数**

2+3.0 结果为5.0，**浮点数**

类型的转换规则以后讲解

3. 运算符的优先级和结合性

- 数学运算的基本法则：先乘除、后加减
- C语言中的运算亦如此：每个操作符都有一个优先级，并有相应的结合性。

➤ 当表达式中包括多个运算符时，将按“优先级”规则解释表达式的含义。

如： $1+2*3$ 等价于 $1+(2*3)$

➤ 当一个操作数两侧的运算符优先级别相同时，则按“结合性”规则处理。

如： $1+2-3$ 等价于 $(1+2)-3$

对表达式优先级和结合性的解释相当于给表达式加括号, 使得表达式能够按照正确的组合逻辑执行计算。

如: $1+2*3$ 等价于 $1+(2*3)$

如: $1+2-3$ 等价于 $(1+2)-3$

45个

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	() [] > . 间接引用结构体成员	圆括号 下标 间接引用结构体成员	左结合
2	! ~ ++、-- +、- (数据类型) &、* sizeof	逻辑非 按位取反 自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*, /, %	乘、除、求余数	左结合
4	+, -	加、减	左结合
5	<<, >>	左移、右移	左结合
6	<, > >=, <=	大于、小于 大于等于、小于等于	左结合
7	==, !=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10		按位或	左结合
11	&&	逻辑与	左结合
12		逻辑或	左结合
13	?:	条件	右结合
14	=, +=, -=, *=, /=, %=, >>=, <<=, &=, ^=, =	赋值	右结合
15	,	逗号	左结合

2.6.2 算术运算

- **算术运算**：对应数学上的加、减、乘、除等运算。
- **算术运算符**：

加法：+ 如：3+6

减法：- 如：6- 4

乘法：* 如：3*8

除法：/ 如：8 / 5

求余：% 如：7 % 4 值为3

正值：+ 如：+3

负值：- 如：-5



注意乘、除符号
和数学上的差异



注意同样符号
的不同语义

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	() [] > . 间接引用结构体成员	圆括号 下标 间接引用结构体成员	左结合
2	! ~ ++、-- +、- (数据类型) &、* sizeof	逻辑非 按位取反 自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*, /, %	乘、除、求余数	左结合
4	+, -	加、减	左结合
5	<<, >>	左移、右移	左结合
6	<, > >=, <=	大于、小于 大于等于、小于等于	左结合
7	==, !=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10		按位或	左结合
11	&&	逻辑与	左结合
12		逻辑或	左结合
13	?:	条件	右结合
14	=, +=, -=, *=, /=, %=, >>=, <<=, &=, ^=, =	赋值	右结合
15	,	逗号	左结合

单目运算符

说明事项:

(1) 对于+、-、*、/

- ◆ 当两个操作数都是整数的时候，运算结果是整数。
- ◆ 有一个操作数是浮点数，运算结果就是浮点数。 类型提升所致

如：整数和整数相除，结果为整数（舍去小数部分）

如： $-5/3 \Rightarrow -1$ $1/2 \Rightarrow 0$

整数和浮点数相除，结果为浮点数

如： $-5/2.0 \Rightarrow -2.5$ $1.0/2 \Rightarrow 0.5$

(2) %的操作数必须为整数

$5\%2 \Rightarrow 1$



$5.0\%2$



例2.7 求出所有的水仙花数

“水仙花数” 是一个三位数，其各位数字的立方和等于该数本身。

如， **153**: $153 = 1^3 + 5^3 + 3^3$ 。

- ◆ 找出“水仙花数”的关键是怎样从一个三位数中分离出百位数、十位数和个位数。
- ◆ 使用什么运算？

设该数为x，则

(1) 求x的个位数: $x \% 10$

(2) 怎么求x的十位数、百位数？

内容先讲

1) 赋值运算： =

如： `x = 5;`

2) 相等关系的判断运算： ==

如： `x == 5`

3) 选择逻辑： if语句

如： `if(x == 5)`

`x = x + 1;`

`else`

`x = x - 1;`

4) 自增运算: ++

如: `int i = 0;`

`i++;`

/*i等于1, 相当于i=i+1;*/

5) 循环逻辑:

for语句:

如: `for( i=1; i<10; i++)`

`printf( "i=%d\n" ,i*i);`

while语句:

如: `i=1;`

`while(i<10) {`

`printf( "i=%d\n" ,i*i);`

`i++;`

`}`

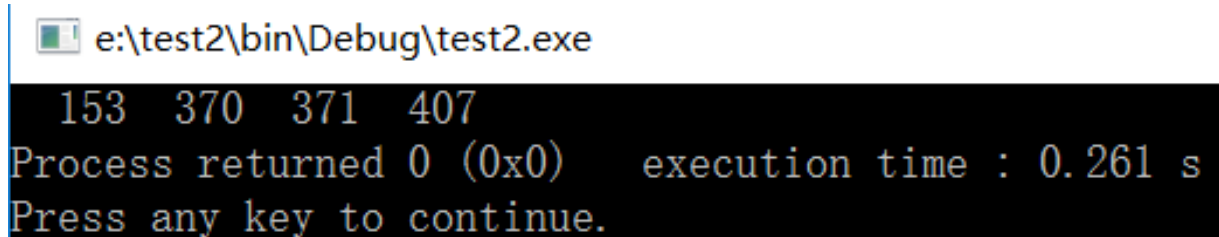
```
for( i=1; i<10; i=i+1)
    printf( "i=%d\n" ,i*i);
```

```

#include <stdio.h>
int main(void)
{
    int x, i, j, k;                                /* x是三位数, i,j,k分别是组成x的三个数字 */
    for (x=100; x<=999; x=x+1)                    /* 从100开始循环到999 */
    {
        i = x / 100;                               /* 分解百位数 */
        j = (x - i * 100) / 10;                   /* 分解十位数 */
        k = x % 10;                                /* 分解个位数 */
        if (x == i*i*i+j*j*j+k*k*k)               /* 测试是否是水仙花数 */
            printf("%5d", x);                       /* 输出 */
    }
    return 0;
}

```

\\源程序\ex2_7.c



```

e:\test2\bin\Debug\test2.exe
153  370  371  407
Process returned 0 (0x0)   execution time : 0.261 s
Press any key to continue.

```

2.6.3 关系运算

◆ 关系运算用于比较两个数的大小，

注：关系运算是个“**问句**”，如果两个操作数的关系成立，则结果为真，否则结果为假。

◆ 有6个关系运算符：

>	大于	如 $5 > 3$
>=	大于等于	如 $x >= y$
<	小于	如 $5 < 3$
<=	小于等于	如 $x <= y$
==	等于	如 $x == y$
!=	不等于	如 $x != y$

注意和日常数学符号写法的不同：

$\geq \rightarrow >=$

$\leq \rightarrow <=$

$= \rightarrow ==$

$\neq \rightarrow !=$

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	() [] > . 间接引用结构体成员	圆括号 下标 间接引用结构体成员	左结合
2	! ~ ++、-- +、- (数据类型) &、* sizeof	逻辑非 按位取反 自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*, /, %	乘、除、求余数	左结合
4	+, -	加、减	左结合
5	<<, >>	左移、右移	左结合
6	<, > >=, <=	大于、小于 大于等于、小于等于	左结合
7	=, !=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10		按位或	左结合
11	&&	逻辑与	左结合
12		逻辑或	左结合
13	?:	条件	右结合
14	=, +=, -=, *=, /=, %=, >>=, <<=, &=, ^=, =	赋值	右结合
15	,	逗号	左结合

■ 关系运算的结果：

- 若关系成立，结果值为1 （代表 “真” ）
- 若关系不成立，结果值为0 （代表 “假” ）

注：一般，关系运算、逻辑运算的结果为bool值，即true或者false。但C语言中没有真正意义的逻辑值true/false，而是用数值0表示false，数值1表示true。

【事实上，任何非0值都视为true】

所以关系表达式的值本质上为int型，false时结果为0，true时结果为1。

关系运算举例

根据变量说明，给出下列表达式的值。

int x=4, y=3, z=2;

char c= 'a' ;

注：字符量参加运算时，其值等于它的ASCII值

(1) $c == \underline{'A'} + 32$

【1】 //关系表达式，判定是否相等

(2) $\underline{c+1} != 'b'$

【0】 //关系表达式，判定是否不等

(3) $x-y \leq 10$

【1】 //关系表达式，判定是否小于等于

(4) $\underline{z=x>y}$

【1】 //赋值表达式，z等于x>y的逻辑值

(5) $\underline{x>y}>z$

【0】 //等价于 $(x>y)>z$

(6) $\underline{x>y}<z$

【1】 //等价于 $(x>y)<z$

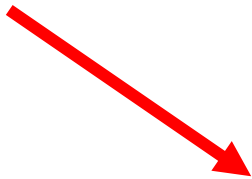
左结合

注意:

- ◆ 数学上判断 x 是否在区间 $[a,b]$ 中时, 习惯写成:

$$a < x < b$$

但C程序中表达式 “ $a < x < b$ ” 的含义与上面的数学表示含义不同, 因此应写成: $a < x \ \&\& \ x < b$



$a < x < b$ 等价于 $(a < x) < b$

注意:



(2) 将关系运算符`==` 误写成赋值运算符`=`，可能会因为运行时的逻辑错误而导致不正确的结果。——程序员很容易犯的错误!

例如: `if (grade == 'A') printf("Very Good! ");`

`/*当成绩的等级为A等时, 输出Very Good!*/`

而: `if (grade = 'A') printf("Very Good!");`

`/*不论成绩的等级是几等, 总输出Very Good!*/` 为什么?

2.6.4 逻辑运算

◆ **逻辑运算**表示操作对象的**逻辑关系**。

◆ 有3个逻辑运算符：

!	逻辑非
&&	逻辑与
	逻辑或

表达式形式：e1&&e2 或 e1||e2

e1	e2	&&	
0	0	0	0
0	非0	0	1
非0	0	0	1
非0	非0	1	1

&& 和 || 的真值表

表达式形式：!e

e	!
0	1
非0	0

! 的真值表

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	() [] > . 间接引用结构体成员	圆括号 下标 间接引用结构体成员	左结合
2	! ~ ++、-- +、- (数据类型) &、* sizeof	逻辑非 按位取反 自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*, /, %	乘、除、求余数	左结合
4	+, -	加、减	左结合
5	<<, >>	左移、右移	左结合
6	<, > >=, <=	大于、小于 大于等于、小于等于	左结合
7	==, !=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10		按位或	左结合
11	&&	逻辑与	左结合
12		逻辑或	左结合
13	?:	条件	右结合
14	=, +=, -=, *=, /=, %=, >>=, <<=, &=, ^=, =	赋值	右结合
15	,	逗号	左结合

单目运算符

- ◆ 逻辑运算的操作数可以是0和任何非0的数值

0: false 非0: true

- ◆ 用逻辑运算符将关系表达式或逻辑量连接起来的式子称为逻辑表达式。

如: `c >= 'a' && c <= 'z'`

- ◆ 逻辑表达式的值: 1 (“真”) 或 0 (“假”)
- ◆ 逻辑表达式的类型: int, 同于关系运算

- 熟练掌握C语言的关系运算符和逻辑运算符后，可以巧妙地表示一个复杂的条件。

(1) 判断整数a是否是偶数：

$!(a\%2)$ 或 $a\%2==0$ 【例2.8】

(2) 判断字符c是否是英文字母：

$c>='a' \ \&\& \ c<='z' \ || \ c>='A' \ \&\& \ c<='Z'$ 【例2.9】

(3) 判断某一年year是闰年还是平年：

判别条件：如果某一年的年份能被4整除但不能被100整除，或者能被400整除，那么该年份是闰年。

$!(year\%4) \ \&\& \ year\%100 \ || \ !(year\%400)$ 【例2.10】

【例2.10 编程判断某年是否是闰年】

短路法则： 编译程序在处理含有 $\&\&$ 、 $\parallel$ 表达式时，往往采用优化方式以提高速度，即：

- $e1\&\&e2$ ：一旦发现 $e1=0$ ，不再计算 $e2$
- $e1\parallel e2$ ：一旦发现 $e1=1$ ，不再计算 $e2$

例如， $x \geq 0.0 \&\& \text{sqrt}(x) \leq 7.7$

若 $x=-1$ ，则结果为0，并且不会报错

但若写成， $\text{sqrt}(x) \leq 7.7 \&\& x \geq 0.0$

若 $x=-1$ ，则报错，因为不能对负数求平方根

2.6.5 自增和自减运算

- ◆ **++** : 自增, ++x 相当于 $x = x + 1$, 使自身的值+1
- ◆ **--** : 自减, --x 相当于 $x = x - 1$, 使自身的值-1
- ◆ ++和--有前缀式和后缀式
 - 前缀式: 把++或--放在变量的前面, 如: ++x, --x
 - 后缀式: 把++或--放在变量的后面, 如: x++, x--

设 `int x = 10, y;`

`y = ++x;` 结果: y等于11, x等于11

`y = --x;` 结果: y等于9, x等于9

`y = x++;` 结果: y等于10, x等于11

`y = x--;` 结果: y等于10, x等于9

注意前缀式与后缀式的区别

- ◆ 前缀式：先执行对操作数的自增(自减)运算，然后使用该操作数的新值参与其他运算
- ◆ 后缀式：先用操作数的旧值参与别的运算，然后再对它作自增(自减)运算

设 `int x = 10, y;`

`y = ++x;` 结果：y等于11, x等于11

`y = --x;` 结果：y等于9, x等于9

`y = x++;` 结果：y等于10, x等于11

`y = x--;` 结果：y等于10, x等于9

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	() [] > . 间接引用结构体成员	圆括号 下标 间接引用结构体成员	左结合
	! ~	逻辑非 按位取反	
2	++、-- +、- (数据类型) &、* sizeof	自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*, /, %	乘、除、求余数	左结合
4	+, -	加、减	左结合
5	<<, >>	左移、右移	左结合
6	<, > >=, <=	大于、小于 大于等于、小于等于	左结合
7	==, !=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10		按位或	左结合
11	&&	逻辑与	左结合
12		逻辑或	左结合
13	?:	条件	右结合
14	=, +=, -=, *=, /=, %=, >>=, <<=, &=, ^=, =	赋值	右结合
15	,	逗号	左结合

单目运算符

1) 自增自减运算只能作用于变量上

如: $5++$,  对常量不能“自增”

$(a+b)--$  对表达式不能“自增”

2) 单目运算符的优先级都是2级，结合性为右结合.

如: $-i++$ 相当于 $-(i++)$

若 $i=3$, 则该表达式结果为 -3 , i 为 4

如: $y=-i++;$

例2.11 统计输入正文的字符数和行数

分析：

- 正文是一行行字符组成的字符序列，每一行是以换行符为结束标志的一串字符。
- 输入正文以Ctrl+Z（DOS系统）或Ctrl+D（UNIX系统）为结束标志（称为文件尾）。
- 接收输入的字符可以使用标准库函数`getchar()`。
- `getchar`遇文件结束标志时返回`EOF`。（`EOF`是在头文件`<stdio.h>`定义的符号常量，其值为-1）。

设计思路：

- (1) **数据上**：定义两个整型变量`nc`和`nl`，分别用于统计正文的字符数和行数；
- (2) **处理逻辑上**：设计一个循环，用`getchar`函数不断从键盘接收输入的字符。
 - 若得到的字符是`EOF`，则退出循环，输出必要信息并结束程序；
 - 否则，字符数`nc`加1，并且若为换行符 `'\n'`，则行数`nl`加1；

注：换行符也是一种字符。

```
/*统计输入正文的字符数和行数。*/
```

```
#include <stdio.h>
```

```
#define EOF -1
```

```
int main(void)
```

```
{
```

```
    int c, nc, nl;
```

```
    printf("Input a text end of ctrl+z:\n"); /* 提示语 */
```

```
    nc = nl = 0;
```

```
    while ((c=getchar()) != EOF)
```

```
    {
```

```
        ++nc;
```

```
        /* 字符数自增1, 可以用nc++代替 */
```

```
        if (c == '\n')
```

```
            ++nl;
```

```
            /* 行数自增1, 可以用nl++代替 */
```

```
    }
```

```
    printf("nc=%d,nl=%d\n", nc, nl);
```

```
    return 0;
```

```
}
```

例2.12 计算 $1+2+3+\dots+n$, n 从键盘输入

分析：对指定的 n ，累加 $1\sim n$ 之间的整数

设计思路：

- 定义整型变量 n ，从键盘输入 n 的值；
- 定义和值 sum ，置 sum 的初值为0；
- 定义整型变量 i ，让 i 从1循环到 n ，并把 i 累加到 sum 上。
- 最后输出 sum 的值。

处理逻辑：循环， i 从1循环到 n ， $sum=sum+i$

注：这里 i 称为循环变量或循环控制变量

■ [\源程序\ex2_12.c](#)


```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int n;
```

```
    int i, sum;                                /* i是循环变量, sum是累加器变量, 存放和值 */
```

```
    printf( "Input your number:" );
```

```
    scanf("%d", &n);
```

```
    i = 1;                                     /* 循环变量初始化为1 */
```

```
    sum = 0;                                  /* 累加器初始化为0 */
```

```
    while (i <= n)                             /* 循环求和 1+2+...+n */
```

```
        sum += i++;                            /* 等价于 sum=sum+i, i++; */
```

```
    printf("sum=%d\n", sum);                  /* 输出和值 */
```

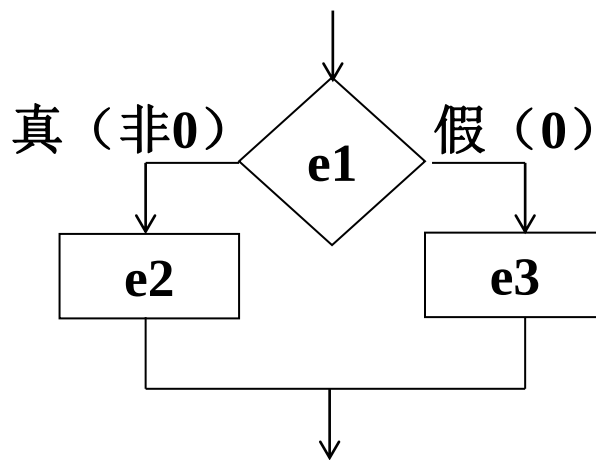
```
    return 0;
```

```
}
```

2.6.6 条件运算 ? :

- ◆ **条件运算符**是C语言中**唯一**的一个**三目运算符**。
- ◆ 条件运算表达式的一般形式: $e1 ? e2 : e3$

执行逻辑:



例: 1) $x=10; y=x>9? 100: 200$ /* y: 100 */

2) $x = a>0 ? 1 : a < 0 ? -1 : 0$ 等价于

$x = a > 0 ? 1 : (a < 0 ? -1 : 0)$

右结合性

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	() [] > . 间接引用结构体成员	圆括号 下标 间接引用结构体成员	左结合
2	! ~ ++、-- +、- (数据类型) &、* sizeof	逻辑非 按位取反 自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*, /, %	乘、除、求余数	左结合
4	+, -	加、减	左结合
5	<<, >>	左移、右移	左结合
6	<, > >=, <=	大于、小于 大于等于、小于等于	左结合
7	==, !=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10		按位或	左结合
11	&&	逻辑与	左结合
12		逻辑或	左结合
13	三目运算符 ?:	条件	右结合
14	=, +=, -=, *=, /=, %=, >>=, <<=, &=, ^=, =	赋值	右结合
15	,	逗号	左结合

例2.13 后缀式++(或--)表达式举例

```
int a=1,b=0;
```

(1) $b++ + b++$

表达式值为0, b为2

(2) $a--\&\&a$

表达式值为0, a为0

(3) $b++ ? b : -b$

表达式值为-1, b为1

为什么有的变量用原值，有的变量用更新后的值？

序列点的问题！

序列点

- ◆ 后缀++(或--)计算延迟的终止点称为**序列点**。
 - 在序列点之前，用原值，序列点之后，该操作数是更改后的新值。
- ◆ 下列条件出现序列点：
 - (1) **&&、||、?:**或**,** 运算符
在这些运算符的**第一个操作数之后**。
如：**a--&&a** 、 **b++ ? b : -b**
 - (2) **完整表达式结束时**，即表达式语句、return语句中的表达式、if、switch或循环语句中的条件表达式（包括for语句中的每个表达式）之后。
如：**b++ + b++**

2.6.7 赋值运算

1. 简单的赋值运算

含义：将右侧“表达式”的值
赋给左侧的变量



- ◆ 赋值运算符： =
- ◆ 赋值表达式： <变量> = <表达式>
- ◆ 赋值表达式的值：左操作数的值
- ◆ 赋值表达式的类型：左操作数的类型

左值 (lvalue)：赋值运算符左侧的标识符，要有“实体”空间

- 变量可以作为左值，间接访问的指针变量也可以
- 表达式不能作为左值，如：a+b
- 常量也不能作为左值，如：5

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	<code>()</code> <code>[]</code> <code>></code> <code>.</code>	圆括号 下标 间接引用结构体成员	左结合
2	<code>!</code> <code>~</code> <code>++</code> 、 <code>--</code> <code>+</code> 、 <code>-</code> (数据类型) <code>&</code> 、 <code>*</code>	逻辑非 按位取反 自增、自减 取正、取负 强制类型转换 取地址、间接引用	右结合

■ 右结合性

如: `a=b=c=3;` 等价于 `a= (b= (c=3) ) ;`

7	<code>=</code> 、 <code>!=</code>	等于、不等于	左结合
8	<code>&</code>	按位与	左结合
9	<code>^</code>	按位异或	左结合
10	<code> </code>	按位或	左结合
11	<code>&&</code>	逻辑与	左结合
12	<code> </code>	逻辑或	左结合
13	<code>?</code> <code>:</code>	条件	右结合
14	<code>=</code> 、 <code>+=</code> 、 <code>-=</code> 、 <code>*=</code> 、 <code>/=</code> 、 <code>%=</code> 、 <code>>>=</code> 、 <code><<=</code> 、 <code>&=</code> 、 <code>^=</code> 、 <code> =</code>	赋值	右结合
15	<code>,</code>	逗号	左结合

2. 复合赋值运算

复合赋值运算： 赋值运算符 = 和另一双目运算符的组合

如： $+=$, $-=$, $*=$, $/=$, $\%=$

注：大多数双目运算符都可以和 “=” 一起构成复合赋值运算符。

含义： $x \text{ op} = y$ 等价于 $x = x \text{ op} y$ 。

如： $i += 2$ 等价于 $i = i + 2$

$y /= x + 10$ 等价于 $y = y / (x + 10)$

$x * = k = m + 5$ 等价于 $x = x * (k = m + 5)$

但 $s[i++] += 1$ 和 $s[i++] = s[i++] + 1$ 不等价 !

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	() [] > . .	圆括号 下标 间接引用结构体成员	左结合
2	! ~ ++、-- +、- (数据类型) &、* sizeof	逻辑非 按位取反 自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*/、/、%	乘、除、求余数	左结合
4	+、-	加、减	左结合
5	<<、>>	左移、右移	左结合
6	<、> >=、<=	大于、小于 大于等于、小于等于	左结合
7	=、!=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10		按位或	左结合
11	&&	逻辑与	左结合
12		逻辑或	左结合
13	? :	条件	右结合
14	=、+=、-=、*=、/=、%=、>>=、<<=、&=、^=、 =	赋值	右结合
15	,	逗号	左结合

2.6.8 逗号运算

■ C语言中，逗号也是一种运算符：

- 逗号运算符： ,
- 逗号运算表达式的一般形式： $e_1, e_2, \dots, e_n$
- 结合性：左结合，即从左往右计算，先计算 e_1 ，再计算 e_2 ，...
- 逗号表达式的值和类型：与最后一个 e_n 的值和类型相同

◆ 其中每个 e_i 是其它形式的表达式；
◆ 整个逗号表达式可以看作是几个表达式用 ' , ' 连结而成的新表达式



例： (1) $x=(i=4, i\%3)$

表达式的值为：1, x为1

(2) $x=i=4, i\%3$

表达式的值为：1, x为4

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	() [] > . .	圆括号 下标 间接引用结构体成员	左结合
2	! ~ ++、-- +、- (数据类型) &、* sizeof	逻辑非 按位取反 自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*/、/、%	乘、除、求余数	左结合
4	+、-	加、减	左结合
5	<<、>>	左移、右移	左结合
6	<、> >=、<=	大于、小于 大于等于、小于等于	左结合
7	=、!=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10	逗号运算符的优先级最低		左结合
11			左结合
12			左结合
13	? :	条件	右结合
14	=、+=、-=、*=、/=、%=、>>=、 <<=、&=、^=、 =	赋值	右结合
15	,	逗号	左结合

注意:

- ◆ 并不是任何地方出现的逗号都是逗号运算符。
- ◆ 只有**作为语句出现的逗号**才构成逗号运算表达式。

例如：函数参数也是用逗号来间隔的（一般是分隔符）

```
printf( "%d,%d" , 2, 3);           /* 输出：2, 3 */
```

对比： **printf("%d,%d" , (2, 3), 3);** /* 输出：3, 3 */



逗号表达式

例2.14 输入一串数字字符，将其转换为一个十进制整数赋值给x。

分析：

- ◆ **输入**：输入一串数字字符有多种方式可以做到。我们采用下面的方式：用`getchar()`函数连续接收输入的多个数字字符。
- ◆ **程序结构**：由于要输入一连串的字符（多个、反复），所以应该用循环语句实现。
- ◆ **数字串怎么转换成整数？（十进制）**
 - **一个数字字符-'0'**就是它对应的数值，如： $0 == '0' - '0'$ ， $1 == '1' - '0'$ 等。
 - 多位数的构造：**每个数码符各自转换成对应数值，然后根据进位制求和**即可，如： $123 == (1*10+2)*10+3$

程序描述：

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    char c;
```

```
    int x;          /* 定义整型变量x，用于存放转换的整数 */
```

```
    for (x=0,c=getchar(); c>='0'&& c<='9'; c=getchar()) /* 遇非数字字符结束循环 */
```

```
        x = 10 * x + c - '0';          /* c-'0': 将数字字符转换成对应的一位整数 */
```

```
    printf( "x=%d\n" , x);
```

```
    return 0;
```

```
}
```

\源程序\ex2_14.c

10/8/2020

$$12345 \\ = (((1*10 + 2)*10 + 3)*10 + 4)*10 + 5$$

```
e:\test2\bin\Debug\test2.exe
123456
x=123456

Process returned 0 (0x0)   execution time : 3.730 s
Press any key to continue.
```

2.6.9 sizeof运算

sizeof 是一个单目运算符，用于求一种数据类型或一个数据对象存储时所需的字节数。

有两种形式：

(1) **sizeof** (类型名)

给出指定的数据类型的数据占用的存储字节数

(2) **sizeof** 表达式

给出表达式结果的类型占用的存储字节数

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	() [] > . ~	圆括号 下标 间接引用结构体成员 逻辑非 按位取反	左结合
sizeof是优先级很高的单目运算符			
	&、*	取地址、间接引用	
	sizeof	数据长度	
3	*/、/、%	乘、除、求余数	左结合
4	+、-	加、减	左结合
5	<<、>>	左移、右移	左结合
6	<、> >=、<=	大于、小于 大于等于、小于等于	左结合
7	=、!=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10		按位或	左结合
11	&&	逻辑与	左结合
12		逻辑或	左结合
13	? :	条件	右结合
14	=、+=、-=、*=、/=、%=、>>=、<<=、&=、^=、 =	赋值	右结合
15	,	逗号	左结合

sizeof举例

假设int类型占用2字节，则：

`sizeof(long)` /* 值为4 */

`double x;` `sizeof x` /* 值为8 */

`int a[10];` `sizeof (a)` /* 值为20 */

说明：a是一个有10个元素的整型数组，每个元素占2个字节，所以共占20个字节

`int a=10,b=10;` `sizeof (a+b)` /* 值为2 */

说明：虽然a+b的值是20，但该值是整型数，sizeof(a+b)是求该结果的类型的长度，所以为2。

分析：如果是sizeof a + b，则值又是多少？

sizeof 是一个常量表达式

- 即使是 `sizeof(a+b)`，也没必要先求 `a+b` 的值，再求 `sizeof(a+b)` 的大小。
 - 因为只要知道操作数类型，就知道结果的类型，`sizeof(a+b)` 的值就可求出。所以 `sizeof(a+b)` 不用放到运行态计算。
- 其计算是在编译时执行的：在编译时就可确定表达式的类型，就可计算 `sizeof(a+b)` 的值，而运行时不再对这个 (`sizeof`) 表达式进行运算。

例如: `short x=1;`

```
sizeof(++x);           /* x不递增， 仍为1 */
```

```
printf( "x=%d\n" , x); /*输出是什么? */
```

2.7 位运算符和位运算表达式

- 字节是C语言最小的存取单位，但最小的可操纵单元是位
- 位运算是施加在位上的运算
- C语言有6个位运算符：
 - ~ (求反)
 - & (按位与)
 - | (按位或)
 - ^ (按位异或，或按位加)
 - << (左移)
 - >> (右移)

所有运算符的优先级和结合性规则

优先级(从高到低)	运算符	运算符名称	结合性
1	() [] > . !	圆括号 下标 间接引用结构体成员 逻辑非	左结合
2	++、-- +、- (数据类型) &、* sizeof	自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*/、/、%	乘、除、求余数	左结合
4	+、-	加、减	左结合
5	<<、>>	左移、右移	左结合
6	<、> >=、<=	大于、小于 大于等于、小于等于	左结合
7	=、!=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10		按位或	左结合
11	&&	逻辑与	左结合
12		逻辑或	左结合
13	? :	条件	右结合
14	=、+=、-=、*=、/=、%=、>>=、<<=、&=、^=、 =	赋值	右结合
15	,	逗号	左结合

单目运算符

2.7.1 按位求反 (~)

含义：单目操作数，对操作数的每个二进制位逐位取反，
即0变1、1变0。

注：即使位运算能作用在“位”上，但操作数还是以字节为基本单位组织的，没有独立的“位”数据。

例如：

二进制表示：	00000000	00000101
		
~后的二进制表示：	11111111	11111010

2.7.2 按位与、或、异或运算 (&, |, ^)

双目操作数，（各字节）对应的两位之间进行与、或、

异或操作

运算规则

与:	或:	异或:
0&0=0	0 0=0	0 ^ 0=0
1&0=0	1 0=1	1 ^ 0=1
0&1=0	0 1=1	0 ^ 1=1
1&1=1	1 1=1	1 ^ 1=0

表达式	二进制表示	十六进制值	说明
x	01101000 11010001	0x68d1	待处理数据
mask	11111111 00000000	0xff00	逻辑尺/掩模
x & mask	01101000 00000000	0x6800	将x的低字节置为0，保留高字节
x mask	11111111 11010001	0xffd1	将x的低字节保留，高字节置为1
x ^ mask	10010111 11010001	0x97d1	将x的低字节保留，高字节翻转

2.7.3 左移和右移运算 (<<和>>)

- $e \ll n$: 将e的值向**左移n位**, 低n位填入0。

如: $(00001011)_2 \ll 2 = (00101100)_2$

注: 左移1位相当于该数值乘以2, 如上: 9 vs 36。

- $e \gg n$: 将e的值向**右移n位**, 而高n位分情况讨论:

--若e是无符号类型数据, 则高n位填入0;

如: $(00001011)_2 \gg 2 = (00000010)_2$

--若e是有符号类型数据, 一些机器填入0 (即 “逻辑移位”),
而另一些机器则填入符号位 (称为 “算术移位”),

如: $(11110101)_2 \gg 2 = (11111101)_2$ 。

注: 右移1位相当于该数值除以2, 如上。

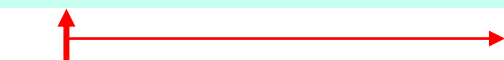
移位运算符的例子



表达式	二进制表示	值	行为
a	00000000 00001011	11	
b	00000000 00001111	15	
-a	11111111 11110101	-11	对a进行负号运算（求-a的补码）
a<<2	00000000 00101100	44	a左移2位（相当于乘4）
b>>3	00000000 00000001	1	b右移3位（相当于除8）
-a>>2	11111111 11111101	-3	-a右移2位（填入符号位1，而其他机器可能是0）

例2.15 写一个表达式，取一个16位整数x从第m位开始向右的n位，并使其向右端靠齐。注：这里一个整数的各个二进制位从右至左依次编号为第0位、第1位、第2位、.....。

1 0 1 1 1 0 0 0 1 0 1 0 1 1 1 0



设m=12, n=8

(1) 将要取出的那n位移到最右端

$x \gg (m - n + 1)$

?????10111000101

(2) 设计一个逻辑尺：低n位全为1，其余位全为0

$\sim(\sim 0 < n)$

0000000011111111

(3) 将上面二者进行&的运算

$x \gg (m - n + 1) \& \sim(\sim 0 < n)$

0000000011000101

2.7.4 位运算符应用举例

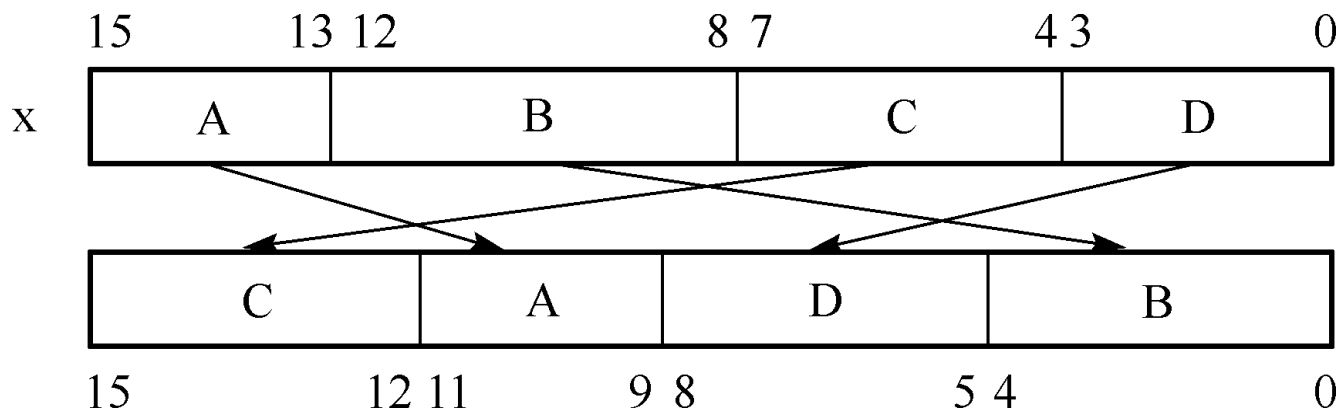
例2.16 可以把表示21世纪日期的日、月和年3个整数压缩成1个16位的整数。写一个表达式，实现压缩存储日、月和年。

- 分析：21世纪日期具有20yymmdd的形式，其中日、月和年分别是3个整数，日有31个值，月有12个值，年有100个值（不考虑20两个数码），所以可以在一个整数中用5b表示日（0~31），用4b表示月（0~15），用7b表示年（0~127）。

15	11 10	7 6	0
日	月	年	

- 计算表达式： $\text{day} < < 11 \mid \text{month} < < 7 \mid \text{year}$

例2.17 简单加密示例。将一个短整型数 x 分成4个长度不等的部分： A (3b)、 B (5b)、 C (4b) 和 D (4b)，然后将它们按照 $CADB$ 的顺序重新拼凑在一起，实现对其加密的功能。写一个表达式，要求其值为 x 的密文。



A (3b) 、 B (5b) 、 C (4b) 和 D (4b)

分析:

(1) 设计4个逻辑尺, 分别用于取出A、 B、 C和D 4个部分:

- A的逻辑尺: 0xE000; B的逻辑尺: 0x1F00;
C的逻辑尺: 0x00F0; D的逻辑尺: 0x000F。

(2) 取出A、 B、 C和D, 并移到新的位置。

(3) 将各部分用位或运算重新组装在一起。

计算表达式为:

$(x \& 0xE000) \gg 4 \mid (x \& 0x1F00) \gg 8 \mid (x \& 0x00F0) \ll 8 \mid (x \& 0x000F) \ll 5$

CADB

2.7.5 打印整数各位的软件工具

例 2.19 函数PrintBit的功能是用屏蔽字显示int型数据的每一个二进制位。

```
/* 按位显示 int */
```

```
#include <limits.h>
```

```
void PrintBit(int x)
```

```
{  
    int i = 0;  
    while (x <= 0)
```

```
    {  
        putchar(x & 1 ? '1' : '0');  
        x <<= 1;  
        if (i % CHAR_BIT == 0) putchar(' ');  
        i++;  
    }  
}
```

```
    putchar ( ! ( x & mask ) ?  '0' : '1'); /*只测试当前x的最高位*/
```

```
    x <<= 1; /*每做完一次测试，左移1位*/
```

```
    if ( ! ( i % CHAR_BIT ) && i < n ) putchar(' ');
```

```
}
```

```
}
```

2.7.5 打印整数各位的软件工具

例 2.19 函数PrintBit的功能是用屏蔽字显示int型数据的每一个二进制位。

```
/* 按位显示 int */
#include<limits.h>
void bit_print(int x)
{
    int i;
    int n=sizeof(int) * CHAR_BIT;    /* CHAR_BIT 在limits.h中定义 */
    int mask = 1 << (n-1);           /* 逻辑尺mask=100.....0 */
    for ( i=1; i<=n; ++i) {
        putchar ( ! ( x & mask ) ?  '0' : '1'); /*只测试当前x的最高位*/
        x<<=1;                               /*每做完一次测试，左移1位*/
        if ( ! ( i % CHAR_BIT ) && i<n ) putchar(' ');
    }
}
```

设计思路：设计一个最高位为1，其余各位为0的逻辑尺mask，用mask来依次测试x的各位。这要用到位左移操作，将x的各位依次移到最高位进行测试。

2.8 类型转换

- C语言允许双精度、单精度、整型及字符数据的混合运算，如：10 + '1' + 6.5
- 在有混合类型的数据参加运算时，先要对操作数进行数据类型转换，类型一致后再进行计算。分四种情形：
 - ◆ 整数提升
 - ◆ 算术转换
 - ◆ 赋值转换
 - ◆ 强制转换

2.8.1 整数提升

当表达式中有char、unsigned char、short和unsigned short类型的数据，则首先将之转换成int或unsigned。这种转换称为“整数提升”。

转换规则：如果原始类型的表示范围小于int，则转换成int，否则转换成unsigned。

如：

设 x 的类型是 `unsigned short`，值为1。则，

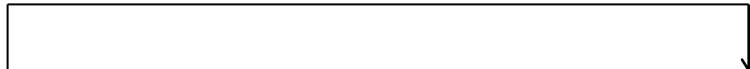
- 如果 `short` 类型的范围小于 `int` 类型（如 `short` 是2字节、`int` 是4字节），则表达式 $(-x)$ 的类型为 `int`，值为-1；
—— x 提升为 `int`，再进行负号运算。
- 如果 `short` 类型的范围等于 `int` 类型（都是2字节），则表达式 $(-x)$ 的类型为 `unsigned`，值为一个大正数。
—— x 只提升为 `unsigned`。

2.8.2 算术转换

对双目运算符的两个操作数，**首先对每个操作数独立进行整数提升，提升之后如果两个操作数类型不相同，再进行算术转换。**

算术转换的规则：

char / short → int → unsigned → long → unsigned long
→ float → double → long double



简而言之： 值域较窄的类型向值域较宽的类型转换。

如：设有 int i; float f; 则 i+f 中，先把操作数i转换为float类型，然后进行计算，最后表达式i+f的类型也为float。

例 2.20 自学

例 2.21 求表达式 $-1 < 1U$ 的值。

分析：真？ 假？ 假！

解析：因为-1为int（设为2个字节，采用补码表示，在计算机中为16个“1”），而1U为unsigned。因此根据类型转换的规则，-1被转换成unsigned，大小为65535。所以最终表达式的值为0（假）。

注：类型转换不改变机器数各位的取值，只是将数据按照新的数据类型进行解释。

2.8.3 赋值转换

- 如果赋值运算符两侧的类型不一致，则进行**赋值转换**。
- **赋值转换的规则：**

将右操作数的类型转换为左操作数的类型。

—— 赋值表达式结果的类型最终由赋值运算符左操作数的类型决定。

例如： `short s=5; double d=2.9;`

则， `s = d` `/*把d转换为short，再赋给s。值为2*/`

`d = s` `/*把s转换为double，再赋给d。值为5.0*/`

2.8.4 强制类型转换

- 整数提升、算术转换、赋值转换都是**隐式转换**，自动完成；
- **强制类型转换**是**显示转换**。
- 强制类型转换的形式： **(类型名) 操作数**

例如，

注：不论何种情况，强制类型转换**只改变对象的值的类型**（产生一个中间结果），对对象本身不产生影响。

(double) i /* 将**i的值**强制转换成double类型，但i的类型保持不变*/

(long)('a' -32) /* 'a' 被自动转换成int，相减的结果被强制转换为long*/

(float)x+y /*等价于((float)x)+y*/

(double)x=10 /*错误*/

2.9 枚举类型

- ◆ char、int、float、double是C语言**内置**的数据类型，称为系统的**预定义数据类型**。
- ◆ 基于基本数据类型，用户可以定义一些扩展的类型，这些类型不是C语言本身所具有的，称为**用户自定义数据类型**。
 - **枚举类型就是一种用户自定义类型。**

注：枚举类型是一种类型的声明，其地位等同于int等类型。

◆ 为什么要定义枚举类型？

例：用数字0~6表示星期日~星期六。

为了使用方便，可以有如下 **“独立”** 符号常量定义：

```
#define SUN 0
#define MON 1
#define TUE 2
#define WED 3
#define THU 4
#define FRI 5
#define SAT 6
```

```
int day;
day = MON;
day = 7;
```



➤ 可用！

➤ 但不好用

1) 定义时书写啰嗦

2) 这些符号常量独立定义，整体上没有形成一个有机的集合来表现
“为表示星期几而专门设置的一些量” 的这一层含义。

这里，`day`只是一个普通的int型变量，取值不受前面定义的符号常量的任何限制

枚举常量的值:

(1) 在未指定值的缺省情况下，默认是第一个枚举常量的值为0，以后的值依次递增1。

如: `enum week { SUN, MON, TUE, WED, THU, FRI, SAT };`

枚举常量的值: 0 1 2 3 4 5 6

(2) 允许指定其中一个或多个枚举常量的值，而未指定值的枚举常量的值比前面的值大1。

如: `enum sizes { SMALL, MEDIUM=10, BIG, TOO_BIG=20 };`

枚举常量的值: 0 10 11 20

◆ 枚举类型定义的形式：

```
enum [枚举名] { 标识符[ = 常量表达式], 标识符[ = 常量表达式], ...};
```

其中，枚举名是该枚举类型的名字；

标识符是集合中枚举量的名字，称为枚举常量；

常量表达式用于给这些枚举常量指定初值；

- 枚举类型是一种用户自定义类型。
- 枚举类型适用于取值为整数、取值范围有限、枚举量个数不多的应用场合。

注：enum 后面也可以不出现枚举名。

如：enum { WIN, LOSE, TIE, ERROR};

2.9.3. 枚举变量的声明

- ◆ 用enum声明的枚举类型声明该类型的变量，称为枚举变量。

(1) 在定义枚举类型的同时说明枚举变量

如：enum color { RED, GREEN, BLUE} c1, c2;

(2) 先定义枚举类型，再说明枚举变量

如：enum color { RED, GREEN, BLUE} c1;

enum color c2;

或者 enum color { RED, GREEN, BLUE};

enum color c1, c2;

■ 使用枚举变量

- (1) 操作时，枚举变量的值被解释成int型整数，因此它可以出现在整数允许出现的任何地方。但枚举变量的值域仅限于枚举集合中列举出来的值，不在该范围内的值将出错。
- (2) 枚举变量值的输入和输出都只能是整数。

如： `enum color { RED, GREEN, BLUE} c1, c2;`

`c1=BLUE;` `/* 等价于 c1=2; */`

`printf( "%d" ,c1);` `/* 输出2, 而不是BLUE */`

`scanf( "%d" ,&c2);` `/* 可输入0、1、2, 但不能输入RED、
GREEN、BLUE等*/`

下面的语句是错误的:

```
enum color { RED, GREEN, BLUE} c1, c2;
```

- ◆ `c1=3;` /* 变量c1的值域为: 0、1和2 */
- ◆ `printf( "%s" , GREEN );` /* 输出错误的结果, 而不是GREEN */
 - 正确的使用方法:

```
if(c1==RED)    printf( "Red" );  
else if(c1==GREEN)  printf( "Green" );  
else if(c1==BLUE)   printf( "Blue" );
```

例2.22 用枚举变量day控制循环，输出存储在数组
weekName中的英文星期名。(课下编程)

分析：要求输出星期的英文名，但枚举变量的值是一个int整数，不是字符串，不能直接输出，怎么办？

方法：定义一个以星期英文名为元素的字符串数组，而以枚举值作为数组下标来索引该字符串数组中的星期名，之后就可以用printf输出了。

■ 源程序\ex2_22.c

```

/* 简要描述： 用枚举变量控制循环，输出存储在数组weekName中的英文星期名。 */
#include <stdio.h>

enum week { SUN, MON, TUE, WED, THU, FRI, SAT };

int main(void)
{
    enum week day;
    char * weekName[ ] = {
        "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday",
        "Friday", "Saturday"
    };
    for (day=SUN; day<=SAT; day++)
        printf("\n%3d\t%s", day, weekName[day] );

    return 0;
}

```

```

0      Sunday
1      Monday
2      Tuesday
3      Wednesday
4      Thursday
5      Friday
6      Saturday
Process returned 0 (0x0)   execution time : 4.916 s
Press any key to continue.

```

2.10 新增数据类型

C99标准增加了支持64位的整数类型long long，还增加了布尔类型和复数类型。CodeBlocks编译器能够较好地支持C99标准，本节中的例子程序均由CodeBlocks运行通过。

2.10.1 long long类型

long long类型的长度至少应为64位。和其他整型类型一样，有带符号和无符号两种：

- signed long long : 数值范围为 $-2^{63} \sim 2^{63}-1$
 - unsigned long long : 数值范围为 $0 \sim 2^{64}-1$ 。
- long long类型的常量用后缀“LL”或“ll”表示。

注：在所有整型类型中，类型long long的值域最宽，因此，C99标准的自动转换所遵循的转换方向为：

...→unsigned long→long long→unsigned long long→float→...

例2.23 输出斐波那契（FIBONACCI）数列的前60个数。

分析：

- FIBONACCI数列是由1和1 开始，之后的数是它前面两数的和，即 1, 1, 2, 3, 5, 8,
- 由于FIBONACCI数列从大约第45个数起就超过了long类型的范围，为避免溢出，应将结果变量说明为long long类型，从输出结果可以看到，它的长度为8字节。

源程序\ex2_23.c

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    long long f1 = 1, f2 = 1;
```

```
    int i;
```

```
    printf("sizeof(long long )=%d\n", sizeof(f1));
```

```
    for (i=1; i<=30; i++)
```

```
    {
```

```
        printf("f(%d)=%l64d\nf(%d)=%l64d\n", (2*i-1), f1, i*2, f2);
```

```
        f1 = f1 + f2;
```

```
        f2 = f2 + f1;
```

```
    }
```

```
    return 0;
```

```
}
```

2.10.2 布尔类型

- C语言中可以用任何整数类型表示布尔值，0表示假，所有非0表示真。布尔表达式为假时值为0，为真时值为1。例如，`i=(a<b)`，在a小于b时对整型变量i赋值1；在a不小于b时对变量i赋值0。
- C99标准引入了真正的布尔类型`_Bool`，`_Bool`类型长度为1，只能保存数值0和1（分别表示“false”与“true”）。虽然也可以使用其他整数类型表示布尔值，但如果C语言实现支持C99标准，那么使用`_Bool`类型更清晰。
- 另外，C99标准为了让C和C++兼容，增加了一个头文件`stdbool.h`。里面定义宏名`bool`为`_Bool`的同义词，定义`false`与`true`分别为0和1。因此，只要在源文件中包含`stdbool.h`这个头文件，就可以在C语言里像C++那样使用`bool`定义布尔类型变量，通过`true`和`false`对布尔变量进行赋值。
- 将任意非零值赋值给`_Bool`类型，都会先转换为1，表示真。将零值赋值给`_Bool`类型，结果为0，表示假。

例2.24 捕鱼和分鱼问题

A、B、C、D、E五人在某天夜里合伙捕鱼，到第二天凌晨时都疲惫不堪，于是各自找地方睡觉。A第一个醒来，他将鱼分为五份，把多余的一条扔掉，拿走自己的一份。B第二个醒来，也将鱼分为五份，把多余的一条扔掉，拿走自己的一份。C、D、E依次醒来，也按同样的方法拿鱼。问他们合伙捕了多少条鱼？

分析：总共将所有的鱼进行了5次平均分配，每次分配的策略是相同的，即扔掉一条后剩下的分为五份，然后拿走自己的一份，余下其他的四份。

假定鱼的总数为 X ，则 X 可以按照题目的要求进行五次分配： $X-1$ 后可被5整除，余下的鱼为 $4*(X-1)/5$ 。若 X 满足上述要求，则 X 就是题目的解。

```
#include <stdio.h>
#include <stdbool.h>
int main(void)
{
    int n, i, x;
    bool flag = true;                /*flag: 控制标记*/
    for (n=6; flag; n++)             /*采用试探的方法, 令试探值n逐步加大*/
    {
        for (x=n, i=1; flag&& i<=5; i++) /*判断是否可以按照题目要求进行5次分配*/
        {
            if ((x-1)%5 == 0)
                x = 4 * (x - 1) / 5;
            else
                flag = false;         /*若不能分配则设置标记flag=0, 退出分配*/
        }
        if(flag)
            break;                   /*若分配过程正常结束则找到结果退出试探过程*/
        else
            flag=true;               /*否则继续试探下一个数*/
    }
    printf("Total number of fish caught=%d\n", n);

    return 0;
}
```

2.10.3 复数类型

- C99标准新增了2个复数类型关键字：

_Complex 和 **_Imaginary**

其中，_Imaginary表示只有一个实数组成的纯虚数类型（即bi）。由于复数的实部和虚部均为实数，在说明复数类型时，需用浮点类型名进一步指定对应实数类型，因此复数或纯虚数各有以下3种类型：

- float _Complex、double _Complex、long double _Complex
- float _Imaginary、double _Imaginary和long double _Imaginary

- 复数类型 `_Complex` 表示为对应实数类型的二元数组，第1个元素表示复数的实数部分，第2个元素表示复数的虚数部分。而复数类型 `_Imaginary` 是实数部分总为0的纯虚数类型。
- C99标准还引入了头文件 `complex.h`，其中定义 `complex` 宏为关键字 `_Complex` 的同义词，定义 `imaginary` 宏为关键字 `_Imaginary` 的同义词，定义 `_Complex_I` (或 `I`) 为复数类型的虚数常量表达式，取值为虚数单位 i ，或。因此，复数型常量用包含了虚数常量 `_Complex_I` (或 `I`) 的浮点型常量表达式来表示，如 $1.0+2.0*I$ ， $1.0+2.0*i$ ， $4.5*i$ 等。
- 复数的运算必须要有相应的库函数支持。在C99标准中，复数的库函数很多，如三角函数、双曲函数、指数函数和其他函数等，这些函数的原型在头文件 `complex.h` 中说明。

复数类型转换规则

- (1) 一个复数类型转换成另一个复数类型时，实数和虚数浮点数成员按照浮点类型转换规则分别转换。
- (2) 从Complex类型转换为_Imaginary类型时，放弃实数部分；从_Imaginary类型转换为_Complex类型时，将实数部分设置为0。
- (3) 将一个实数（整数或浮点数）转换成复数类型时，该实数转换成复数值的实数部分，而虚数部分为0。
- (4) 将一个复数类型转换成实数（整数或浮点数）时，虚数部分放弃。

例2.25 求两个复数的乘积。

```
#include <stdio.h>
#include <complex.h>
int main(void)
{
    double _Complex a, b, c;
    printf("The size of a is %d\n", sizeof(a));
    a = 2 + 3 * I;
    b = 4 + 5 * I;
    c = a * b;
    printf("c=%f+%f*I\n", creal(c), cimag(c));
    return 0;
}
```

说明：程序中定义了3个双精度的复数a、b和c，第一条printf函数调用语句输出复数a所需的内存大小，它为double的2倍。接着，给复数变量a和b分别赋一个复数常量，复数a和b的乘积赋值给复数变量c。函数creal和cimag分别为取得复数的实部和虚部。

本章小结

- 记号是程序中具有语义的最基本组成单元，共分5类：标识符、关键字、常量、运算符和标点符号。
 - 标识符是程序员用于对变量和函数等命名的符号。
 - 关键字有明确的含义，不能被用作普通标识符。
 - 常量包括字符常量、字符串常量，以及各种类型的整型常量、浮点型常量，必须熟练掌握各种类型常量的表示方法。
- C语言提供的基本数据类型有：字符型、整型、短整型、长整型，以及单精度、双精度和高精度浮点型。
 - 在C99标准中增加了布尔类型、长长整型、复数类型等。
 - 枚举类型是用标识符命名的整型常量的集合，其中的标识符实际上是自动设置值的符号常量。

- C语言提供了很多运算符，除通常的算术、关系、逻辑和赋值运算符外，还有一些C语言特有的运算符：自增、自减、条件、逗号、复合赋值、sizeof和位运算符。
 - 自增自减运算符有前缀式和后缀式，但效果可能不同，需要特别注意。
 - 位运算符使程序员可以访问字节或字中的实际二进制位，要重点掌握位运算符的应用。
- 每个表达式都有一个值和类型，运算符的优先级和结合性规则决定了表达式求值的顺序，在表达式的计算过程中，会涉及类型之间的转换问题，有自动类型转换和强制类型转换两种规则。